

Ten clean builds. Identical binaries.

PASS · Controlled reproducibility check

Every executable, static library and object file matched run 01 byte for byte. All staged assets matched by SHA-256. No binary changes were found.

10/10

clean builds completed

1

unique executable SHA-256

0

compiled files changed

0

asset files changed

Executable fingerprint

86b4f7cb870b2b74a204b30f82ea802523a5445b17be6a2113845d7dcab291f9
4,575,320 bytes · 4.363 MiB · ELF64 / x86-64
GNU build ID: b4c33c4ecbc4a3a7e11c0ce98fefcd221d012b56f

The pre-existing build/deathward also matches the clean-build executable byte for byte.

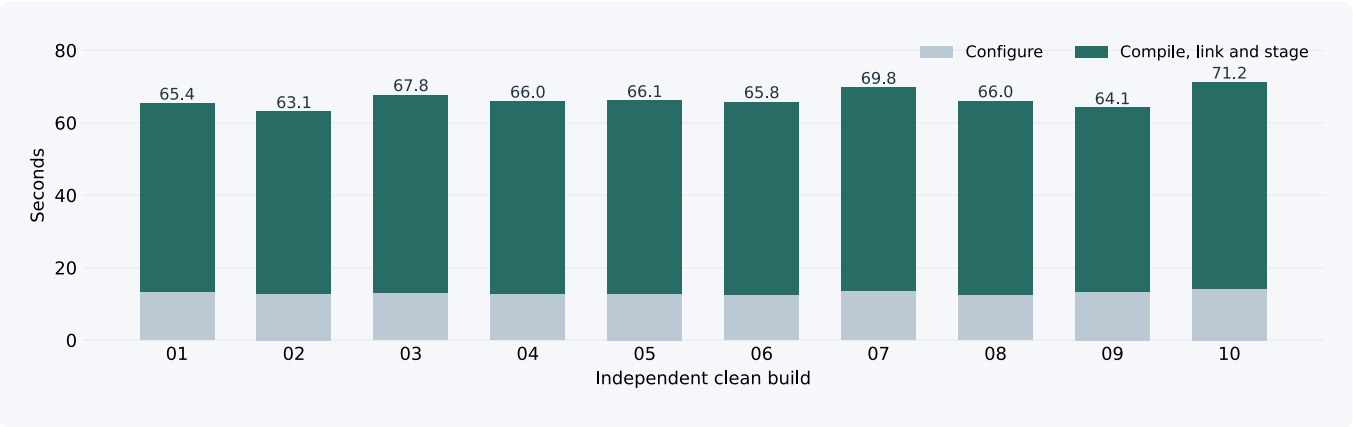
What was compared in every run

ARTIFACT CATEGORY	FILES PER RUN	COMPARISON	RESULT
Linux executable	1	SHA-256 + direct byte comparison	1 unique hash
Static libraries	6	SHA-256 + direct byte comparison	Identical
Compiled object files	82	SHA-256 + direct byte comparison	Identical
Staged models, textures, audio, maps and fonts	188	SHA-256 by relative path	Identical
ELF sections	32	Contents + section layout	Identical

89 compiled files and 188 staged asset files per run. Assets total 213,313,420 bytes. The game was not modified.

Every run, measured

RUN	CONFIGURE	BUILD	EXECUTABLE SHA-256	CHANGED COMPILED / ASSETS
01	13.37 s	52.03 s	86b4f7cb870b2b74...	0 / 0
02	12.76 s	50.31 s	86b4f7cb870b2b74...	0 / 0
03	13.12 s	54.64 s	86b4f7cb870b2b74...	0 / 0
04	12.69 s	53.27 s	86b4f7cb870b2b74...	0 / 0
05	12.74 s	53.38 s	86b4f7cb870b2b74...	0 / 0
06	12.59 s	53.24 s	86b4f7cb870b2b74...	0 / 0
07	13.63 s	56.14 s	86b4f7cb870b2b74...	0 / 0
08	12.62 s	53.43 s	86b4f7cb870b2b74...	0 / 0
09	13.26 s	50.88 s	86b4f7cb870b2b74...	0 / 0
10	14.10 s	57.11 s	86b4f7cb870b2b74...	0 / 0



Mean build phase: 53.44 s; median: 53.32 s. Bars show configure + build only; archive extraction, hashing and analysis are excluded. Timings are observational.

Libraries match as well as the executable

STATIC LIBRARY	BYTES	SHA-256 (PREFIX)	UNIQUE HASHES
libraylib.a	3,257,134	53bd9fdc2399f1bc	1
libdeathward_cloth.a	6,388	af338a1a22b0a21d	1
libdeathward_core.a	1,867,752	51a0a955dc230780	1
libdeathward_mission_lighting.a	75,196	371d234423033412	1
libdeathward_particles.a	166,942	f69ffd455b376b4d	1
libdeathward_skinning.a	83,230	fb35eaffff344d577	1

What changed, and what it means

There were **0 changed compiled files**. The observable changes occurred outside the binary contents:

Filesystem dates changed

Each run created new files. Their modification times differ, even though their contents and SHA-256 hashes match. A changed date in a file manager does not imply a changed program.

FILE	RUN 01 MODIFIED (UTC)	RUN 10 MODIFIED (UTC)
deathward	2026-10-03T14:24:54.039+00:00	2026-10-03T14:35:05.258+00:00
libdeathward_core.a	2026-10-03T14:24:25.962+00:00	2026-10-03T14:34:29.094+00:00

Build configuration paths changed

The CMake cache, Makefile and compilation database record their run-specific directories. Replacing only each run’s directory prefix makes the four inspected metadata files identical. The linker command is already identical before normalization.

FILE	RAW CONTENT	AFTER REPLACING EACH RUN’S PATH
CMakeCache.txt	Different	Identical
CMakeFiles/deathward.dir/link.txt	Identical	Identical
Makefile	Different	Identical
compile_commands.json	Different	Identical

Build timing and log order can vary

The measured build phase ranges from 50.31 to 57.11 seconds. Timing differences alone are not binary differences. This experiment did not isolate the causes of timing variation. OS caches were not flushed, and parallel compiler scheduling was not controlled, so these measurements are not a cold-cache benchmark.

Why the binaries stayed stable

The observations are consistent with a deterministic build under the tested conditions:

- The source fingerprint remained unchanged, the dependency archive was hash-verified, and the compiler/linker settings stayed fixed.
- All 6 archives have zero timestamps and owner/group IDs for their real members. GNU ar’s deterministic mode removes those varying inputs from archive bytes. [GNU ar documentation](#).
- The executable has no DWARF debug sections, and its bytes contain no audit-directory prefix. The fixed source asset path remains embedded.
- The linker command and object/library order are identical. The GNU build ID is also identical; the linker’s default SHA-1 style is content-based. The full-file SHA-256 and direct byte comparisons provide the stronger file-equality check. [GNU ld documentation](#).

Build diagnostics: 16 warning lines in the first build; the same normalized warning set appears in every run. They concern raylib/dependency code and the MAX_MESH_VERTEX_BUFFERS definition. These warnings were recorded, not repaired, in this compilation audit.

Method & scope

1. Start from the same tracked game inputs at commit `f67dc4ef76a6`. Verify their aggregate content fingerprint before and after the experiment.
2. Create a new run directory and a previously nonexistent build directory. Freshly extract the raylib archive whose SHA-256 matches CMake's pinned value.
3. Configure Release with the recorded GCC/G++ toolchain, Unix Makefiles, static libraries, sanitizers off and 4 compiler jobs. Disable compiler launchers and ccache.
4. Build target `deathward`, including its dependencies. Game test executables and the navigation-baking utility are outside this comparison.
5. Hash the executable, every produced object/library and every staged asset. Compare each compiled file directly against run 01's bytes; inspect ELF sections, build IDs and archive headers.
6. Run each executable with `--help` and retain commands, manifests, logs, symbols and complete build trees.

SETTING	RECORDED VALUE
Source commit	f67dc4ef76a633e0fc9328e5403137c827e178ab
C++ compiler	g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
CMake	cmake version 3.22.1
Linker / archiver	GNU ld (GNU Binutils for Ubuntu) 2.38
Platform	Ubuntu 22.04.5 LTS, x86-64, WSL2
Optimization	-O3 -DNDEBUG
Parallelism	4 compiler jobs per run; runs executed sequentially
Source asset mode	DEATHWARD_PACKAGED_ASSETS=OFF (normal Linux development setting)
SOURCE_DATE_EPOCH	None — unset
Prefix remapping / stripping	Neither applied to compared binaries
Finished (UTC)	2026-10-03T14:35:06.828476+00:00

Limits of the conclusion

The result establishes repeatability for this source, toolchain, flags and host while changing build/dependency extraction directories and compilation time. It does not establish equality across different compiler versions, machines, Debug builds or Windows/WASM. The main source checkout path was fixed. Its asset path is embedded in the executable, so relocating that checkout is a separate test. Shared system libraries and GPU drivers were not rebuilt. This audit checks compiled output, not gameplay correctness or runtime performance.

Recommendation

No reproducibility repair is needed for the tested Linux Release configuration. Keep the pinned dependency and record compiler versions for releases. Before promising reproducibility across machines, test a relocated source checkout, packaged asset mode, compiler versions and archive packaging separately. GCC provides path remapping for compiler-recorded paths; the explicitly embedded asset-directory string must also be handled. [GCC path-remapping documentation](#).

Evidence & reproduction

Selected executable sections

EXECUTABLE SECTION	BYTES	RESULT
.text	3,444,390	Identical
.rodata	325,432	Identical
.data	9,168	Identical
.symtab	159,984	Identical
.strtab	229,865	Identical
.note.gnu.build-id	36	Identical
.comment	45	Identical

All section layouts were compared, including the size of the zero-initialized .bss section. No .debug* sections are present in this Release executable. Raw binaries were compared without stripping or prefix normalization.

Run the experiment again

```
python3 scripts/check-linux-reproducibility.py \  
  --runs 10 --jobs 4 \  
  --output artifacts/linux-reproducibility-new-run  
  
python3 scripts/report-linux-reproducibility.py \  
  artifacts/linux-reproducibility-new-run/results.json \  
  --output docs/reports/linux-reproducibility-new-run.html
```

The runner needs the cached raylib 5.5 source archive (or --raylib-archive PATH), normal Linux build dependencies and sufficient space for all build trees. It refuses to overwrite an existing evidence directory. The report renderer additionally uses Matplotlib.

Evidence locations and integrity

Full build trees, logs, manifests, symbols and ELF inspections:

/home/kambo/workspace/DeathWard/artifacts/linux-reproducibility-2026-10-03

Source-content manifest digest:

19678e21ed00b3319a2f6a339393e8bdf446347853eb969859bb493e1c6e6d6d

Verified raylib source archive SHA-256:

aea98ecf5bc5c5e0b789a76de0083a21a70457050ea4cc2aec7566935f5e258e

Raw results.json SHA-256:

2ca8b5236fa7143c5287a21f4977f5df7b2df3be54e99ef743a2e35c30de5ee6

Companion JSON contains the environment, per-run comparisons and compiled-file fingerprints; CSV contains the full executable hashes and timings. The report works offline. External links point only to the GNU toolchain documentation supporting the explanations.